MiniLang Compiler — Project Report

Student Name	Kushagar Sood
Roll Number	235805198
Course Title	Compiler Design
Semester	VI
Faculty Name	Mrs. Shreya Banerjee
Submission Date	12.04.2026
Project	Design a DSL and Build its Compiler Pipeline
Option	Option 1 — Programming Language as described in assignment
Language Name	MiniLang
Implementation Language	Python 3

---

1. Abstract

This report presents the design and implementation of a complete compiler pipeline for **MiniLang**, a small imperative programming language supporting integer variables, arrays, arithmetic and relational operators, control flow (`if-then-else`, `while-do`), and I/O operations (`read`, `write`). The compiler is implemented in Python and consists of six modular phases: **Lexical Analysis**, **Parsing** (Recursive Descent), **Semantic Analysis**, **Intermediate Code Generation** (Three-Address Code), **Code Generation** (Stack-Machine Assembly), and **Listing File Output**. The compiler produces a listing file that numbers each source line, interleaves error messages below the corresponding lines, and generates target code only when the source is error-free. Each phase is independently testable and produces meaningful error messages with line numbers, allowing the compiler to continue analysis even in the presence of errors.

---

2. Introduction

2.1 Motivation

Compilers are fundamental to computer science — they bridge the gap between human-readable source code and machine-executable instructions. Building a compiler from scratch

provides deep insight into how programming languages are processed, validated, and translated.

2.2 Objectives

1. Design the lexical specification, grammar, and semantics for a small programming language (MiniLang).
2. Implement each compiler phase as a separate, modular Python module.
3. Ensure robust error handling — the compiler must report errors with line numbers and continue processing.
4. Generate three-address intermediate code and stack-machine assembly as target code.
5. Produce a listing file showing numbered source lines with inline errors.

2.3 Language Features

MiniLang supports:

Feature	Syntax
Integer variables	`var x, y : integer;`
Arrays	`var arr : array[10] of integer;`
Assignment	`x := 5;`
Arithmetic operators	`+`, `-`, `*`, `/`
Relational operators	`<`, `>`, `<=`, `>=`, `=`, `<>`
Logical operators	`and`, `or`, `not`
If-then-else	`if expr then stmt else stmt`
While loop	`while expr do stmt`
Compound statements	`begin ... end`
Read (multiple)	`read(x, y, z)`
Write (multiple)	`write('text', expr, 42)`
Strings with escapes	`'hello\tworld\n'`
Comments	`! comment to end of line`
Procedure calls	`proc_name(args)`

---

3. Description of the Implemented Compiler

3.1 Project Structure

```
DSL/
├── minilang/
│   ├── __init__.py        # Package initializer
│   ├── tokens.py          # Phase 1: Token definitions
│   ├── lexer.py           # Phase 1: Lexical analyzer
│   ├── ast_nodes.py       # Phase 2: AST node definitions
│   ├── parser.py          # Phase 2: Recursive descent parser
│   ├── semantic.py        # Phase 3: Semantic analyzer
│   ├── irgen.py           # Phase 4: IR code generator
│   ├── codegen.py         # Phase 5: Target code generator
│   └── listing.py         # Phase 6: Listing file generator
├── examples/
│   ├── test1.ml           # Sample MiniLang program
│   ├── fib.ml             # Fibonacci example
│   ├── test_errors.ml     # Lexical error test
│   ├── test_parse_err.ml  # Parse error test
│   └── test_semantic_err.ml  # Semantic error test
└── main.py                # CLI entry point
```

3.2 Compiler Pipeline — Block Diagram

```
┌──────────────────────────┐
│    Source Program        │
│      (.ml file)          │
└──────────────────────────┘
            │
            ▼
┌──────────────────────────┐        ┌──────────────────────┐
│  Phase 1: Lexical        │──────▶│   Token Stream       │
│    Analyzer              │        │ (type, value, ln)    │
│  (tokens.py, lexer.py)   │        └──────────────────────┘
└──────────────────────────┘
            │
            ▼
┌──────────────────────────┐        ┌──────────────────────┐
│  Phase 2: Syntax         │──────▶│  Abstract Syntax     │
│    Analyzer (Parser)     │        │   Tree (AST)         │
│  (ast_nodes.py,          │        └──────────────────────┘
│   parser.py)             │
└──────────────────────────┘
            │
            ▼
┌──────────────────────────┐        ┌──────────────────────┐
│  Phase 3: Semantic       │──────▶│  Symbol Table +      │
│    Analyzer              │        │  Type Checking       │
│  (semantic.py)           │        └──────────────────────┘
└──────────────────────────┘
            │
```

```
                 │
                 ▼
┌──────────────────────────┐      ┌──────────────────────┐
│  Phase 4: Intermediate   │──────►│  Three-Address       │
│    Code Generator        │      │  Code (TAC)          │
│  (irgen.py)              │      └──────────────────────┘
└──────────────────────────┘
                 │
                 ▼
┌──────────────────────────┐      ┌──────────────────────┐
│  Phase 5: Code           │──────►│  Stack-Machine       │
│    Generator             │      │  Assembly            │
│  (codegen.py)            │      └──────────────────────┘
└──────────────────────────┘
                 │
                 ▼
┌──────────────────────────┐      ┌──────────────────────┐
│  Phase 6: Listing        │──────►│  Listing File        │
│    File Generator        │      │ (Lines + Errors +    │
│  (listing.py)            │      │  Target Code)        │
└──────────────────────────┘      └──────────────────────┘
```

3.3 Phase 1 — Lexical Analyzer (`tokens.py`, `lexer.py`)

Purpose: Transform raw source text into a stream of tokens.

Token Table

Category	Tokens
Keywords	`program`, `var`, `integer`, `begin`, `end`, `if`, `then`, `else`, `while`, `do`, `read`, `write`, `procedure`, `function`, `not`, `and`, `or`, `of`, `array`, `return`
Identifiers	letter (letter | digit)* — case-insensitive, significant up to 32 chars
Literals	`INTLIT` (integer), `STRLIT` (string with `\n`, `\t`, `\'` escapes)
Operators	`+` `-` `*` `/` `:=` `<` `>` `<=` `>=` `=` `<>` `and` `or` `not`
Delimiters	`(` `)` `[` `]` `;` `:` `,` `.`
Special	`EOF`, `ERROR`

Key design decisions:

- **TokenType Enum:** 40 token categories covering all reserved words, literals, operators, delimiters, and special tokens.
- **Token Dataclass:** Each token carries `(token_type, value, line)`.
- **Case-insensitive identifiers:** All identifiers are folded to lowercase.
- **32-character significance:** Identifiers are truncated to 32 characters.
- **String escape sequences:** `\n`, `\t`, `\'` are handled inside single-quoted strings.
- **Comments:** `!` to end-of-line are skipped.

- **Error recovery:** On illegal characters or unterminated strings, an `ERROR` token is emitted and scanning continues.

Core methods:

Method	Responsibility
`__init__(source)`	Store source, initialize position and line counter
`advance()`	Move forward one character, track newlines
`peek()`	Return current character without advancing
`skip_ws_and_c()`	Skip whitespace and `!`-comments
`read_identifier()`	Read `[a-zA-Z_][a-zA-Z0-9_]*`, truncate, fold case, check reserved words
`read_number()`	Read `[0-9]+`, return `INTLIT`
`read_string()`	Read `'...'` with escape handling; error on unterminated
`read_op_or_delim()`	Handle single and compound operators (`:=`, `<=`, `>=`, `<>`)
`next_token()`	Top-level dispatch
`tokenize()`	Loop `next_token()` until `EOF`

3.4 Phase 2 — Parser (`ast_nodes.py`, `parser.py`)

Purpose: Consume the token stream and build an Abstract Syntax Tree (AST) via recursive descent parsing.

Formal Grammar Rules

```
program             → program id ; declarations compound_stmt .
declarations        → ( var id_list : type ; )*
id_list             → id ( , id )*
type                → standard_type | array [ num ] of standard_type
standard_type       → integer
compound_stmt       → begin stmt_list end
stmt_list           → statement ( ; statement )*
statement           → variable := expression
                    | procedure_statement
                    | compound_stmt
                    | if expression then statement else statement
                    | if expression then statement
                    | while expression do statement
                    | read_statement
                    | write_statement
procedure_statement → id ( argument_list )
arguments           → ( parameter_list ) | ( )
read_statement      → read ( identifier_list )
identifier_list     → variable , identifier_list | variable
```

```
write_statement     → write ( output_list )
output_list         → output_item | output_item , output_list
output_item         → string | expression
expression          → simple_expr ( relop simple_expr )?
simple_expr         → term ( ( + | - | or ) term )*
term                → factor ( ( * | / | and ) factor )*
factor              → num | string | id | id [ expression ]
                    | id ( argument_list ) | ( expression ) | not factor
```

AST Node classes: `Program`, `VarDecl`, `ArrayType`, `Compound`, `Assign`, `IfStmt`, `WhileStmt`, `ReadStmt`, `WriteStmt`, `ProcCall`, `BinOp`, `UnaryOp`, `Num`, `Str`, `Identifier`, `ArrayAccess`, `FuncCall`.

Error recovery: On unexpected tokens, errors are recorded with line numbers and the parser attempts to continue by skipping the bad token.

3.5 Phase 3 — Semantic Analyzer (`semantic.py`)

Purpose: Walk the AST and perform static checks.

Checks performed:

1. **Symbol table construction** — Variables and arrays are registered with their type, kind (`var` / `array`), size, and declaration line.
2. **Duplicate declaration detection** — `Variable 'a' already declared (first at line 3)`.
3. **Undeclared identifier detection** — `Undeclared identifier 'c'`.
4. **Type checking** — `Type mismatch in assignment: 'integer' := 'string'`.
5. **Array usage validation** — `Array 'b' used without index` and `'a' is not an array`.

Visitor pattern: The analyzer uses `visit_<NodeType>` methods that return the inferred type (`"integer"`, `"string"`, or `"error"`).

3.6 Phase 4 — Intermediate Code Generator (`irgen.py`)

Purpose: Walk the AST and emit three-address code (TAC).

Instruction format: `(op, arg1, arg2, result)`

Key instructions:

Op	Meaning
`:=`	Assignment: `result := arg1`
`+`, `-`, `*`, `/`	Arithmetic: `result := arg1 op arg2`
`<`, `>`, `<=`, `>=`, `=`, `<>`	Comparison: `result := arg1 op arg2`

Op	Meaning
`IFZ`	If arg1 is zero, jump to result (label)
`GOTO`	Unconditional jump to result (label)
`LABEL`	Label definition
`READ`	Read integer into result
`WRITE`	Output arg1
`CALL`	Call procedure arg1 with arg2 parameters
`PARAM`	Push parameter
`[]=`	Array store: `result[arg2] := arg1`
`=[]`	Array load: `result := arg1[arg2]`
`HALT`	Program end

Temporaries: Named `t1, t2, ...` (monotonically increasing counter).
Labels: Named `L1, L2, ...` (monotonically increasing counter).

3.7 Phase 5 — Code Generator (`codegen.py`)

Purpose: Translate three-address code into stack-machine pseudo-assembly.

Stack-machine instruction set:

Category	Instructions
Data	`PUSH <immediate>`, `LOAD <variable>`, `STORE <variable>`
Arithmetic	`ADD`, `SUB`, `MUL`, `DIV`
Comparison	`CMP_LT`, `CMP_GT`, `CMP_LE`, `CMP_GE`, `CMP_EQ`, `CMP_NE`
Logic	`AND`, `OR`, `NOT`
Control flow	`JUMP <label>`, `JUMP_IF_ZERO <label>`, `<label>:`
I/O	`READ_INT`, `WRITE_INT`, `PUSH_STR`, `WRITE_STR`
Arrays	`ALOAD <name>`, `ASTORE <name>`
Program	`HALT`

Translation approach: Each TAC instruction is translated to a sequence of stack operations: - Binary operations push both operands, apply the operation, and store the result. - Assignments load the source and store to the target. - Branches use `JUMP_IF_ZERO` for conditional and `JUMP` for unconditional jumps.

3.8 Phase 6 — Listing File Generator (`listing.py`)

Purpose: Produce a formatted compiler listing.

Output format: 1. Source lines numbered sequentially. 2. Error messages printed with `***` directly below the source line they refer to. 3. A summary line showing total error count. 4. Target code (assembly) only if zero errors.

---

4. Design Methodology

The compiler was developed using a **phase-wise modular approach**. Each phase was implemented and tested independently before integrating into the full pipeline.

Development steps:

1. Token specification and lexer implementation
2. Grammar design and recursive descent parser
3. AST construction and validation
4. Symbol table and semantic checks
5. Three-address code generation
6. Stack-machine code generation
7. Listing file formatting and error integration

Unit tests were created for each phase to ensure correctness before moving to the next stage.

---

5. Results — Snapshots of Each Compiler Phase

5.1 Sample Input Program (`fib.ml`)

```
program fib;
var a, b, temp, i : integer;
begin
  a := 0;
  b := 1;
  i := 10;
  while i > 0 do
  begin
    temp := a + b;
    a := b;
    b := temp;
    i := i - 1
  end;
```

```
  write('Result: ', b)
end.
```

5.2 Phase 1 Output — Lexer (`--phase lexer`)

Command: `python3 main.py examples/fib.ml --phase lexer`

```
Line   Token Type     Value
--------------------------------------------------
1      PROGRAM        'program'
1      IDENTIFIER     'fib'
1      SEMICOLON      ';'
2      VAR            'var'
2      IDENTIFIER     'a'
2      COMMA          ','
2      IDENTIFIER     'b'
2      COMMA          ','
2      IDENTIFIER     'temp'
2      COMMA          ','
2      IDENTIFIER     'i'
2      COLON          ':'
2      INTEGER        'integer'
2      SEMICOLON      ';'
3      BEGIN          'begin'
4      IDENTIFIER     'a'
4      ASSIGN         ':='
4      INTLIT         0
4      SEMICOLON      ';'
...
14     WRITE          'write'
14     LPAREN         '('
14     STRLIT         'Result: '
14     COMMA          ','
14     IDENTIFIER     'b'
14     RPAREN         ')'
15     END            'end'
15     DOT            '.'
16     EOF            ''

No lexical errors.
```

Screenshot:

```
$ python3 main.py examples/fib.ml --phase lexer

Line   Token Type     Value
-----------------------------------------------------
1      PROGRAM        'program'
1      IDENTIFIER     'fib'
1      SEMICOLON      ';'
2      VAR            'var'
2      IDENTIFIER     'a'
2      COMMA          ','
2      IDENTIFIER     'b'
2      COMMA          ','
2      IDENTIFIER     'temp'
2      COMMA          ','
2      IDENTIFIER     'i'
2      COLON          ':'
2      INTEGER        'integer'
2      SEMICOLON      ';'
3      BEGIN          'begin'
4      IDENTIFIER     'a'
4      ASSIGN         ':='
4      INTLIT         0
4      SEMICOLON      ';'
5      IDENTIFIER     'b'
5      ASSIGN         ':='
5      INTLIT         1
5      SEMICOLON      ';'
6      IDENTIFIER     'i'
6      ASSIGN         ':='
6      INTLIT         10
6      SEMICOLON      ';'
7      WHILE          'while'
7      IDENTIFIER     'i'
7      GT             '>'
7      INTLIT         0
7      DO             'do'
8      BEGIN          'begin'
9      IDENTIFIER     'temp'
9      ASSIGN         ':='
9      IDENTIFIER     'a'
9      PLUS           '+'
9      IDENTIFIER     'b'
9      SEMICOLON      ';'
14     WRITE          'write'
14     LPAREN         '('
14     STRLIT         'Result: '
14     COMMA          ','
14     IDENTIFIER     'b'
14     RPAREN         ')'
15     END            'end'
15     DOT            '.'
16     EOF            ''

No lexical errors.
```

5.3 Phase 2 Output — Parser / AST (`--phase parser`)

Command: `python3 main.py examples/fib.ml --phase parser`

```
═══ AST ═══
Program 'fib'
  VarDecl ['a', 'b', 'temp', 'i'] : integer
  Compound
    Assign
      Id 'a'
      Num 0
    Assign
      Id 'b'
      Num 1
    Assign
      Id 'i'
      Num 10
    While
      BinOp '>'
        Id 'i'
        Num 0
    Do
      Compound
        Assign
          Id 'temp'
          BinOp '+'
            Id 'a'
            Id 'b'
        Assign
          Id 'a'
          Id 'b'
        Assign
          Id 'b'
```

```
            Id 'temp'
          Assign
            Id 'i'
            BinOp '-'
              Id 'i'
              Num 1
      Write
        Str 'Result: '
        Id 'b'
```

Screenshot:

5.4 Phase 3 Output — Semantic Analysis (`--phase semantic`)

Command: `python3 main.py examples/fib.ml --phase semantic`

```
═══ Symbol Table ═══
Name             Kind     Type       Size   Line
--------------------------------------------------
a                var      integer    0      2
b                var      integer    0      2
temp             var      integer    0      2
i                var      integer    0      2

No semantic errors.
```

Screenshot:

```
$ python3 main.py examples/fib.ml --phase semantic

ã•ã• Symbol Table ã•ã•ã•
Name            Kind     Type       Size   Line
-----------------------------------------------
a               var      integer    0      2
b               var      integer    0      2
temp            var      integer    0      2
i               var      integer    0      2

No semantic errors.
```

5.5 Phase 4 Output — Intermediate Code (`--phase ir`)

Command: `python3 main.py examples/fib.ml --phase ir`

```
═══ Three-Address Code ═══
#    Op       Arg1           Arg2           Result
-------------------------------------------------------
0    PROGRAM  fib
1    :=       0                             a
2    :=       1                             b
3    :=       10                            i
4    LABEL                                  L1
5    >        i              0              t1
6    IFZ      t1                            L2
7    +        a              b              t2
8    :=       t2                            temp
9    :=       b                             a
10   :=       temp                          b
11   -        i              1              t3
12   :=       t3                            i
13   GOTO                                   L1
14   LABEL                                  L2
15   WRITE    'Result: '
16   WRITE    b
17   HALT
```

Screenshot:

```
$ python3 main.py examples/fib.ml --phase ir

â•¼â•¼â•¼ Three-Address Code â•¼â•¼â•¼
#    Op       Arg1          Arg2          Result
--------------------------------------------------
0    PROGRAM  fib
1    :=       0                            a
2    :=       1                            b
3    :=       10                           i
4    LABEL                                 L1
5    >        i  -Ý         0              t1
6    IFZ      t1                           L2
7    +  -Ý       a              b           t2
8    :=       t2   -Ý                      temp
9    :=       b                            a
10   :=       temp                         b
11   -        i             1              t3
12   :=       t3                           u
13   GOTO        -Ý                        L1
14   LABEL                                 L2
15   WRITE    'Result: '
16   WRITE    b
¤17   HALT
```

5.6 Phase 5 Output — Code Generation (`--phase codegen`)

Command: `python3 main.py examples/fib.ml --phase codegen`

```
═══ Stack-Machine Assembly ═══
; ── program fib ──
  PUSH 0
  STORE a
  PUSH 1
  STORE b
  PUSH 10
  STORE i
L1:
  LOAD i
  PUSH 0
  CMP_GT
  STORE t1
  LOAD t1
  JUMP_IF_ZERO L2
  LOAD a
  LOAD b
  ADD
  STORE t2
  LOAD t2
  STORE temp
  LOAD b
  STORE a
  LOAD temp
  STORE b
  LOAD i
  PUSH 1
  SUB
  STORE t3
```

```
  LOAD t3
  STORE i
  JUMP L1
L2:
  PUSH_STR 'Result: '
  WRITE_STR
  LOAD b
  WRITE_INT
HALT
```

Screenshot:

5.7 Phase 6 Output — Full Listing (Clean Compile)

Command: `python3 main.py examples/fib.ml`

```
═══ Source Listing ═══

     1 | program fib;
     2 | var a, b, temp, i : integer;
     3 | begin
     4 |   a := 0;
     5 |   b := 1;
     6 |   i := 10;
     7 |   while i > 0 do
     8 |   begin
     9 |     temp := a + b;
    10 |     a := b;
    11 |     b := temp;
    12 |     i := i - 1
    13 |   end;
    14 |   write('Result: ', b)
```

```
    15 | end.

── 0 errors. Compilation successful. ──

═══ Generated Code ═══

; ── program fib ──
  PUSH 0
  STORE a
  ...
  WRITE_INT
HALT
```

5.8 Error Handling — Listing with Errors

Command: `python3 main.py examples/test_semantic_err.ml`

Input:

```
! Semantic error test: undeclared vars, type mismatches, duplicate decl
program semantic_test;
var a : integer;
var a : integer;
var b : array[5] of integer;
begin
  a := 10;
  c := 5;
  b := 3;
  b[1] := 'hello';
  a[2] := 1;
  write(a + b)
end.
```

Output:

```
═══ Source Listing ═══

     1 | ! Semantic error test: undeclared vars, type mismatches, duplicate dec
l
     2 | program semantic_test;
     3 | var a : integer;
     4 | var a : integer;
       *** [line 4] Variable 'a' already declared (first at line 3)
     5 | var b : array[5] of integer;
     6 | begin
     7 |   a := 10;
     8 |   c := 5;
       *** [line 8] Undeclared identifier 'c'
     9 |   b := 3;
       *** [line 9] Array 'b' used without index
    10 |   b[1] := 'hello';
       *** [line 10] Type mismatch in assignment: 'integer' := 'string'
```

```
    11 |   a[2] := 1;
       *** [line 11] 'a' is not an array
    12 |   write(a + b)
       *** [line 12] Array 'b' used without index
    13 | end.

── 6 error(s) found. Target code NOT generated. ──
```

Screenshot:

Key observations: - Each error appears directly below the offending source line. - All 6 errors are detected in a single pass (the compiler does not stop at the first error). - Target code is suppressed because errors exist.

5.9 Testing Strategy

The compiler was tested systematically using the following approach:

1. **Phase-by-phase testing** — The `--phase` CLI flag was used to test each phase independently:
2. `--phase lexer` — Verified token stream for all lexeme categories
3. `--phase parser` — Verified AST structure matches expected parse trees
4. `--phase semantic` — Verified symbol table and error detection
5. `--phase ir` — Verified three-address code correctness
6. `--phase codegen` — Verified stack-machine assembly output
7. **Dedicated error test files:**

8. `test_errors.ml` — Deliberate **lexical errors** (unterminated string, illegal `@`)
9. `test_parse_err.ml` — Deliberate **parse errors** (missing semicolons, bad syntax)
10. `test_semantic_err.ml` — Deliberate **semantic errors** (undeclared vars, type mismatches, array misuse)
11. **End-to-end test** — `fib.ml` (Fibonacci computation) tested the full pipeline from source to assembly.
12. **Error recovery verification** — All test files with errors were confirmed to:
13. Report every error with line numbers
14. Continue analysis after each error
15. Suppress target code when errors are present

---

6. Discussion

6.1 Novelties

1. **Fully modular architecture** — Each phase is an independent Python module. Any phase can be tested or replaced without affecting others. The `--phase` CLI flag allows running any individual phase.
2. **Robust error recovery** — All phases continue after errors:
3. The lexer skips illegal characters and unterminated strings.
4. The parser records errors and continues by skipping bad tokens.
5. The semantic analyzer visits all nodes even after type mismatches.
6. **Inline error listing** — Errors are grouped by line number and displayed directly below the offending source line, making it easy to locate and understand each issue.
7. **Multi-item I/O** — `read(x, y, z)` reads multiple variables and `write('text', expr, 42)` outputs multiple items (strings and expressions) in a single statement, matching the grammar specification.
8. **Stack-machine code generation** — The generated assembly uses a clean, well-defined instruction set that could be executed by a simple virtual machine interpreter.
9. **Comprehensive type system foundation** — While MiniLang currently supports only integers, the visitor-based semantic analyzer is designed to be extended to additional types easily.

6.2 Limitations

1. **No procedure/function definitions** — The grammar supports procedure calls, but procedure and function declarations with bodies are not implemented.
2. **Integer only** — No floating-point, boolean, or character types.
3. **No runtime execution** — The generated stack-machine assembly is not executed; a VM interpreter would be needed.
4. **No optimization** — The IR and generated code contain redundant operations.
5. **Single scope** — All variables live in a single global scope.
6. **No standard library** — Only `read` and `write` are built-in.

6.3 Future Enhancements

1. **Procedure and function definitions** — Implement `procedure` and `function` declarations with parameters, local variables, and return values using activation records on the stack.
2. **Boolean type** — Add `true` / `false` literals and boolean variables to enable proper type checking for conditions in `if` and `while` statements.
3. **Virtual machine interpreter** — Build a stack-machine VM that can actually execute the generated assembly code, making the compiler produce runnable programs.
4. **Code optimization** — Implement optimization passes such as:
5. **Constant folding** — Evaluate `3 * 2` at compile time instead of runtime.
6. **Dead code elimination** — Remove unreachable code after `GOTO` or `HALT`.
7. **Peephole optimization** — Simplify `STORE t1 / LOAD t1` to reuse the stack value.
8. **Register allocation** — Map temporaries to a fixed set of registers instead of using unbounded temporaries.
9. **For-loop and nested arrays** — Extend the language with `for` loops and multi-dimensional arrays.

---

7. Conclusion

This project successfully implements a complete compiler pipeline for the MiniLang programming language, covering all six required phases: Lexical Analysis, Parsing, Semantic Analysis, Intermediate Code Generation, Code Generation, and Listing File Output.

The compiler correctly: - Tokenizes source code with support for identifiers (case-insensitive, 32-char significant), integers, strings with escape sequences, compound operators, and comments. - Parses programs using recursive descent and builds a complete AST. - Performs semantic checks including symbol table construction, duplicate detection, undeclared identifier detection, and type checking. - Generates three-address intermediate code with temporaries and labels. - Translates intermediate code to stack-machine pseudo-assembly. - Produces a listing file with numbered source lines, inline error messages, and conditional target code output.

The modular architecture ensures that each phase can be tested independently and extended in the future. The error recovery mechanism across all phases allows the compiler to report multiple errors in a single compilation, significantly improving the developer experience.

How to run:

```
cd ~/DSL
python3 main.py <source_file.ml>                    # full compile (default)
python3 main.py <source_file.ml> --phase <phase>      # individual phase
```

Where `<phase>` is one of: `lexer`, `parser`, `semantic`, `ir`, `codegen`, `compile`.

End of Report